

```
"""
```

```
=====
Coibion-Gorodnichenko predictability test on the ECB SPF
Full replication script -- Carlo Rappenecker, Bachelor Thesis
=====
```

```
This single file reproduces every empirical result in Chapters 5 and 6:
Tables 1-4 and Figures 1-3. It is deterministic (no random seeds).
```

```
INPUTS (place these two raw files next to this script, or edit the paths below):
```

```
  * SPF_ZIP      : ECB SPF microdata, one CSV per round (e.g. "Rates_BA_1.zip")
                   Download: https://www.ecb.europa.eu/stats/ecb\_surveys/
                   survey\_of\_professional\_forecasters/html/all\_data.en.html
  * EUROSTAT_GZ  : Eurostat realized HICP, dataset prc_hicp_aind (.csv.gz)
                   Download: https://ec.europa.eu/eurostat/databrowser/
                   view/prc\_hicp\_aind
```

```
METHOD (fixed-event, calendar-year targets):
```

```
  consensus_t(Y) = mean point forecast across forecasters
  error_t(Y)      = realized(Y) - consensus_t(Y)
  revision_t(Y)   = consensus_t(Y) - consensus_{t-1}(Y)      (consecutive rounds)
  regress error on revision; beta>0 = under-reaction; lambda = beta/(1+beta)
  split: D=1 from 2020Q1 (survey-round date); interaction delta tests H2
```

```
USAGE: python cg_spf_replication.py
```

```
OUTPUT: prints all tables; writes cg_results_table.csv, cg_split_table.csv and
        the three figures as PNG files.
```

```
Requires: python>=3.10, pandas, numpy, statsmodels, matplotlib, pillow
```

```
=====
"""
```

```
import re, glob, os, zipfile, io
import numpy as np
import pandas as pd
import statsmodels.api as sm
import matplotlib
matplotlib.use("Agg")
import matplotlib.pyplot as plt
```

```
# -----
# CONFIGURATION -- edit these two paths to point at your raw data files
# -----
SPF_ZIP      = "Rates_BA_1.zip"
EUROSTAT_GZ  = "Rates_BA_2_csv.gz"
```

```

BREAK          = "2020Q1"                # first post-break survey round
HORIZONS       = [0, 1, 2]                # calendar-year horizons kept (drop ~5y)
H_BASE_LAGS    = 4                        # Newey-West lags (quarterly overlap)

SECTION_MARKERS = ("INFLATION EXPECTATIONS", "CORE INFLATION", "GROWTH EXPECTATIO
                  "EXPECTED UNEMPLOYMENT", "ASSUMPTIONS")

# =====
# 1. PARSE THE ECB SPF MICRODATA
# =====
def read_hicp_block(path):
    """Extract only the HICP inflation section from one SPF round file."""
    with open(path, encoding="utf-8", errors="replace") as f:
        lines = f.read().splitlines()
    try:
        start = next(i for i, l in enumerate(lines) if l.startswith("INFLATION EX
    except StopIteration:
        return None
    stop = next((i for i in range(start + 2, len(lines))
                if any(lines[i].startswith(m) for m in SECTION_MARKERS)), len(lines))
    df = pd.read_csv(io.StringIO("\n".join(lines[start + 1:stop])))
    if not {"TARGET_PERIOD", "FCT_SOURCE", "POINT"}.issubset(df.columns):
        return None
    df = df[["TARGET_PERIOD", "FCT_SOURCE", "POINT"]].copy()
    df["POINT"] = pd.to_numeric(df["POINT"], errors="coerce")
    return df

def load_spf(zip_path, workdir="_spf_tmp"):
    """Unzip all round files and stack the HICP point forecasts in long format."""
    os.makedirs(workdir, exist_ok=True)
    with zipfile.ZipFile(zip_path) as z:
        z.extractall(workdir)
    frames = []
    for path in sorted(glob.glob(os.path.join(workdir, "*.csv"))):
        rnd = os.path.splitext(os.path.basename(path))[0] # e.g. "2019Q4"
        if not re.fullmatch(r"\d{4}Q[1-4]", rnd):
            continue
        d = read_hicp_block(path)
        if d is None:
            continue
        d["survey_round"] = rnd
        frames.append(d)
    m = pd.concat(frames, ignore_index=True)
    # keep only pure calendar-year targets (drop rolling '2020Q1', '2020Aug', ...
    m = m[m["TARGET_PERIOD"].astype(str).str.fullmatch(r"\d{4}")]

```

```

m["target"] = m["TARGET_PERIOD"].astype(int)
m["horizon"] = m["target"] - m["survey_round"].str[:4].astype(int) # years
return m.rename(columns={"FCT_SOURCE": "forecaster_id", "POINT": "point_forec
    ["survey_round", "target", "forecaster_id", "point_forecast", "horizon"]

# =====
# 2. REALIZED EURO-AREA HICP INFLATION (changing composition)
# =====
def load_realized(gz_path):
    eu = pd.read_csv(gz_path)
    r = eu[(eu["unit"] == "Annual average rate of change") &
        (eu["coicop"] == "All-items HICP") &
        (eu["geo"].astype(str).str.startswith("Euro area (EA11")))].copy()
    r["target"] = r["TIME_PERIOD"].astype(int)
    return r.set_index("target")["OBS_VALUE"].astype(float).sort_index()

# =====
# 3. CONSENSUS, FORECAST ERROR AND REVISION
# =====
def build_consensus(micro, method="mean"):
    """Collapse micro forecasts to one consensus per (round, target)."""
    g = micro.groupby(["survey_round", "target"])["point_forecast"]
    consensus = (g.mean() if method == "mean" else g.median()).rename("consensus")
    consensus["horizon"] = micro.drop_duplicates(["survey_round", "target"]).set_
        ["survey_round", "target"]).loc[
        list(zip(consensus["survey_round"], consensus["target"])), "horizon"].val
    return consensus

def build_error_revision(consensus, realized, round_order):
    """Construct forecast error and one-quarter forecast revision."""
    df = consensus.copy()
    rank = {r: i for i, r in enumerate(round_order)}
    df["_rank"] = df["survey_round"].map(rank)
    df = df.sort_values(["target", "_rank"]).reset_index(drop=True)
    df["prev_consensus"] = df.groupby("target")["consensus"].shift(1)
    prev_rank = df.groupby("target")["_rank"].shift(1)
    consecutive = (df["_rank"] - prev_rank) == 1 # exactly one q
    df["revision"] = np.where(consecutive, df["consensus"] - df["prev_consensus"])
    df = df.merge(realized.rename("realized"), left_on="target", right_index=True)
    df["error"] = df["realized"] - df["consensus"]
    return df

# =====

```

```

# 4. ESTIMATORS
# =====

def beta_to_lambda(b):
    return b / (1.0 + b)

def fit_nw(df, lags):
    """OLS error ~ revision with Newey-West (HAC) standard errors."""
    d = df.dropna(subset=["error", "revision"]).sort_values("_rank")
    X = sm.add_constant(d[["revision"]])
    return sm.OLS(d["error"], X).fit(cov_type="HAC", cov_kwds={"maxlags": lags})

def fit_cluster(df):
    """OLS error ~ revision with two-way cluster-robust SE (round, target)."""
    d = df.dropna(subset=["error", "revision"]).copy()
    g = np.column_stack([d["survey_round"].astype("category").cat.codes,
                        d["target"].astype("category").cat.codes])
    X = sm.add_constant(d[["revision"]])
    return sm.OLS(d["error"], X).fit(cov_type="cluster",
                                     cov_kwds={"groups": g, "use_correction": True})

def fit_split(df, break_rank, lags=H_BASE_LAGS):
    """Pooled interaction: error = a + b*rev + g*D + delta*(D*rev)."""
    d = df.dropna(subset=["error", "revision"]).sort_values("_rank").copy()
    d["D"] = (d["_rank"] >= break_rank).astype(float)
    d["Dxrev"] = d["D"] * d["revision"]
    X = sm.add_constant(d[["revision", "D", "Dxrev"]])
    return sm.OLS(d["error"], X).fit(cov_type="HAC", cov_kwds={"maxlags": lags})

def fit_decomposition(df, lags=H_BASE_LAGS):
    """FIRE check: error = a + b1*F_t + b2*F_{t-1}; expect b1>0, b2<0, b1+b2=0."""
    d = df.dropna(subset=["error", "consensus", "prev_consensus"]).sort_values("_rank")
    X = sm.add_constant(d[["consensus", "prev_consensus"]])
    m = sm.OLS(d["error"], X).fit(cov_type="HAC", cov_kwds={"maxlags": lags})
    p_sum = float(m.t_test("consensus + prev_consensus = 0").pvalue)
    return m, p_sum

# =====
# 5. MAIN
# =====

def main():
    micro = load_spf(SPF_ZIP)
    realized = load_realized(EUROSTAT_GZ)
    round_order = sorted(micro["survey_round"].unique(), key=lambda r: (int(r[:4])

    m3 = micro[micro["horizon"].isin(HORIZONS)].copy()
    cons = build_consensus(m3, method="mean")

```

```

er = build_error_revision(cons, realized, round_order).dropna(
    subset=["error", "revision"]).sort_values("_rank").reset_index(drop=True)
break_rank = er.loc[er["survey_round"] == BREAK, "_rank"].min()
h1, h2 = er[er.horizon == 1], er[er.horizon == 2]

print(f"Rounds: {micro.survey_round.nunique()} | forecasters/round "
      f"{micro.groupby('survey_round').forecaster_id.nunique().min()}-"
      f"{micro.groupby('survey_round').forecaster_id.nunique().max()} | "
      f"usable obs: {len(er)} (by horizon {er.horizon.value_counts().sort_index

# ---- Table 2: baseline -----
def rowdict(spec, m, N):
    b, se = m.params["revision"], m.bse["revision"]
    return dict(spec=spec, N=N, beta=round(b, 3), se=round(se, 3),
                t=round(b / se, 2), p=round(m.pvalues["revision"], 3),
                lam=round(beta_to_lambda(b), 3) if b > 0 else np.nan)
baseline = pd.DataFrame([
    rowdict("pooled (h=0,1,2) NW", fit_nw(er, H_BASE_LAGS), len(er)),
    rowdict("pooled two-way cluster", fit_cluster(er), len(er)),
    rowdict("current year h=0 NW", fit_nw(er[er.horizon == 0], H_BASE_LAGS),
    rowdict("1-year-ahead h=1 NW", fit_nw(h1, H_BASE_LAGS), len(h1)),
    rowdict("1-year-ahead h=1 cluster", fit_cluster(h1), len(h1)),
    rowdict("2-year-ahead h=2 NW", fit_nw(h2, 6), len(h2)),
])
baseline.to_csv("cg_results_table.csv", index=False)
print("\n=== Table 2 Baseline ===\n" + baseline.to_string(index=False))

# ---- Table 3: pre/post-2020 split -----
split = []
for name, sub in [("1-year-ahead h=1", h1), ("pooled h=0,1,2", er)]:
    m = fit_split(sub, break_rank)
    split.append(dict(spec=name, beta_pre=round(m.params["revision"], 3),
                    delta=round(m.params["Dxrev"], 3), delta_se=round(m.bse["Dxrev"], 3),
                    delta_p=round(m.pvalues["Dxrev"], 3),
                    beta_post=round(m.params["revision"] + m.params["Dxrev"], 3), N=int(m
split = pd.DataFrame(split)
split.to_csv("cg_split_table.csv", index=False)
print("\n=== Table 3 Pre/post-2020 split ===\n" + split.to_string(index=False)
print(f"post-2020 obs (h=1): {int((h1['_rank'] >= break_rank).sum())}")

# ---- Table 4: robustness -----
med = build_consensus(m3, method="median")
er_med = build_error_revision(med, realized, round_order)
m_med = fit_nw(er_med, H_BASE_LAGS)
h1_nocovid = h1[~h1.survey_round.isin(["2020Q1", "2020Q2", "2020Q3", "2020Q4"]
m_nc = fit_nw(h1_nocovid, H_BASE_LAGS)
m_dec, p_sum = fit_decomposition(er)

```

```

alt_rank = er.loc[er["survey_round"] == "2021Q1", "_rank"].min()
m_alt = fit_split(er, alt_rank)
print("\n=== Table 4 Robustness ===")
print(f" median consensus (pooled beta) : {m_med.params['revision']:.3f} "
      f"(se {m_med.bse['revision']:.3f}, p {m_med.pvalues['revision']:.3f})")
print(f" excl. 2020 COVID rounds (h=1) : {m_nc.params['revision']:.3f} "
      f"(se {m_nc.bse['revision']:.3f}, p {m_nc.pvalues['revision']:.3f})")
print(f" decomposition b1 (F_t) : {m_dec.params['consensus']:.3f} "
      f"(se {m_dec.bse['consensus']:.3f}, p {m_dec.pvalues['consensus']:.3f})")
print(f" decomposition b2 (F_t-1) : {m_dec.params['prev_consensus']:.3f} "
      f"(se {m_dec.bse['prev_consensus']:.3f}, p {m_dec.pvalues['prev_consensus']:.3f})")
print(f" b1+b2=0 test : p {p_sum:.3f}")
print(f" alternative break 2021Q1 (delta): {m_alt.params['Dxrev']:.3f} "
      f"(se {m_alt.bse['Dxrev']:.3f}, p {m_alt.pvalues['Dxrev']:.3f})")
print(" NW lag sensitivity (h=1) beta/p:",
      {L: (round(fit_nw(h1, L).params['revision'], 3),
            round(fit_nw(h1, L).pvalues['revision'], 3)) for L in (1, 2, 4, 6)}

# ---- descriptive statistics (Table 1) -----
print("\n=== Table 1 Descriptives ===")
print(er[["error", "revision"]].describe().round(3).to_string())
print(f"corr(error, revision) = {er['error'].corr(er['revision']):.3f}")

# ---- Figures -----
_make_figures(er, h1, realized, break_rank)
print("\nSaved: cg_results_table.csv, cg_split_table.csv, "
      "cg_exp_vs_realized.png, cg_error_vs_revision.png, cg_target2021_path.png")

def _make_figures(er, h1, realized, break_rank):
    # Figure 1: one-year-ahead consensus vs realized over target years
    exp_by_year = h1.groupby("target")["consensus"].mean()
    yrs = [y for y in range(2000, 2026) if y in realized.index and y in exp_by_year.index]
    plt.figure(figsize=(8, 4.6))
    plt.plot(yrs, [realized[y] for y in yrs], "o-", color="#c44", lw=2, label="Realized")
    plt.plot(yrs, [exp_by_year[y] for y in yrs], "s--", color="#3b6ea5", lw=2, label="One-year-ahead SPF expectations")
    plt.axhline(2.0, color="0.6", lw=.9, ls=":", label="ECB 2% target")
    plt.xlabel("target year"); plt.ylabel("annual HICP inflation (%)")
    plt.title("Euro-area inflation: realized vs. one-year-ahead SPF expectations")
    plt.legend(fontsize=8); plt.tight_layout(); plt.savefig("cg_exp_vs_realized.png")

    # Figure 2: error vs revision, pooled, coloured pre/post 2020
    post = er["_rank"] >= break_rank
    m = fit_nw(er, H_BASE_LAGS)
    xx = np.linspace(er.revision.min(), er.revision.max(), 100)
    plt.figure(figsize=(7.2, 5))
    plt.axhline(0, lw=.8, color="0.7"); plt.axvline(0, lw=.8, color="0.7")

```

```

plt.scatter(er.revision[~post], er.error[~post], s=20, alpha=.6, color="#3b6e
plt.scatter(er.revision[post], er.error[post], s=26, alpha=.8, color="#c44",
plt.plot(xx, m.params["const"] + m.params["revision"] * xx, "k-", lw=2,
         label=f"slope beta = {m.params['revision']:.2f}")
plt.xlabel("consensus forecast revision (pp)"); plt.ylabel("consensus forecas
plt.title("ECB SPF: forecast error vs. revision (HICP, 1999-2025)")
plt.legend(); plt.tight_layout(); plt.savefig("cg_error_vs_revision.png", dpi

# Figure 3: the 2021 target -- consensus path vs realized
t21 = er[er.target == 2021].sort_values("_rank")
plt.figure(figsize=(7.2, 5))
plt.axhline(realized[2021], color="#c44", lw=2, ls="--", label=f"realized 202
plt.plot(range(len(t21)), t21["consensus"], "o-", color="#3b6ea5", label="SPF
plt.xticks(range(len(t21)), t21["survey_round"], rotation=45, ha="right", fon
plt.ylabel("expected 2021 HICP inflation (%)")
plt.title("Forecasters systematically under-predicted 2021 inflation")
plt.legend(); plt.tight_layout(); plt.savefig("cg_target2021_path.png", dpi=1

if __name__ == "__main__":
    main()

```